CMOS RISC-V — Relatório técnico da Fase 2

Versão preliminar 0.3 — 17 de outubro de 2026

Equipe: CMOS. Integrantes, instituição e orientador: **a preencher pela equipe**. Repositório: cmosuea-arch/RV32-multicycle. Referência do RTL e da captura: commit `7ddc6bf`, com snapshot identificado em [R10].

Este documento segue os tópicos 1 a 6 do Guia de Submissão ChampionCHIP, versão 1.0/2026, páginas físicas 2–4 do PDF local [R1]. É um relatório de andamento: contém resultados existentes e identifica entregáveis ainda ausentes. As referências [R1]–[R10] estão ao final da seção 6.

1. Visão Geral do Projeto

1.1 Objetivo e escopo

O projeto desenvolve um processador RISC-V de 32 bits com execução multiciclo, seguindo a arquitetura do Block Guide ChampionCHIP. A meta é implementar o conjunto solicitado pela competição: RV32I, multiplicação Zmmul e extensão customizada Xicrc, totalizando 47 instruções na tabela de avaliação. O núcleo e os módulos RTL são implementações próprias em SystemVerilog. A arquitetura Ibex não foi incorporada; seu uso previsto é apenas referência auxiliar, subordinada ao guia da competição.

O núcleo atual realiza operações aritméticas e lógicas, comparação, desvios, saltos, acesso à memória por byte/meia palavra/palavra e multiplicação. A baseline verificada contempla **41/47 instruções (87,23%)**. Três operações CRC adicionais funcionam sob contrato experimental CMOS, ainda sujeito à confirmação oficial. As três instruções de sistema/sincronização permanecem pendentes de identificação no material oficial disponível no repositório. Esse percentual mede instruções da tabela; não mede a conclusão do projeto.

1.2 Organização do sistema

`riscv_core` reúne PC, IR, banco de registradores, extensão de imediatos, decoder, ALU, comparador de branch, multiplicador, LSU e FSM. O CRC é incluído com `EXPERIMENTAL_XICRC=1`, desabilitado por padrão. `riscv_top` acrescenta o decodificador de endereços e as memórias de instruções e dados.

Uma instrução passa por busca, decodificação, execução e escrita de retorno. Loads/stores acrescentam um estado de memória para acomodar a leitura síncrona da DMEM. O mesmo caminho de endereço atende busca e acesso a dados em ciclos diferentes. O projeto não usa pipeline. A sincronização usa borda de subida; o reset é síncrono, ativo alto, e reinicia o PC em `0x00400000`.

O sistema experimental `simple_system` integra ROM de 2 KiB, RAM de 1 KiB, CRC provisório e saídas de término, sucesso e falha. Ele permite validar um programa próprio tanto em RTL quanto na netlist genérica. A pinagem e a implementação física final da competição continuam em definição.

1.3 Resultado disponível nesta versão

A rodada funcional de referência, registrada em 13/10/2026 no horário de Manaus (14/10/2026 em UTC), aprovou 31 testbenches e lint. O sistema pequeno também passou por síntese lógica e dois boots simulados na netlist. A execução do firmware oficial, os diagramas ChipInventor e o fluxo físico do processador permanecem pendentes. Os resultados são vinculados aos manifestos [R4–R6]. Em 17/10/2026 foi reexecutado somente o firmware próprio para a captura de waveform [R10]; a regressão completa continua referida à rodada histórica identificada.

1.4 Método de desenvolvimento incremental

O trabalho começa pelos contratos: barramentos de 32 bits, sinais de enable, reset, latência das memórias, seleção de operandos, escrita de retorno e mapa de endereços. As escolhas internas são registradas em `docs/architecture/`. A referência oficial tem prioridade; lacunas que afetam conformidade ficam registradas, sem inventar instruções para completar a contagem.

Cada bloco é implementado e submetido a teste autoverificável antes de sua integração. A sequência avança de ALU e registradores para fetch, núcleo mínimo, famílias da ISA, memória e multiplicação. A regressão anterior é repetida a cada integração funcional. Testes, interfaces, diagramas de desenvolvimento e commits permitem relacionar cada requisito ao código e à evidência que o verifica.

Etapa	Incremento desenvolvido	Marco rastreável
Contratos	Interfaces, reset e mapa de memória	baseline-v1
Blocos básicos	ALU, RF, imediatos, PC, IR, IMEM	tags block-*-v1
Primeira execução	Fetch, decode, execute, writeback	core-smoke-v1
Aritmética e imediatos	21 instruções integradas	isa-21-v1
Controle de fluxo	Branches e saltos; 29 instruções	isa-29-v1
Memória	Loads/stores; 37 instruções	isa-37-v1
Zmmul	Quatro multiplicações; 41 instruções	isa-41-v1
CRC experimental	Três operações sob contrato CMOS	commit 05fd114
Sistema pequeno	Firmware e simulação da netlist	commit 2e87e0e

1.5 Papel das ferramentas e da automação

O RTL descreve o circuito. O testbench fornece entradas, gera clock/reset e compara saídas com valores esperados. Os scripts Python calculam referências, geram vetores e programas, invocam as ferramentas e registram evidências. O Makefile organiza esses passos em comandos reproduzíveis; ele delega o trabalho aos scripts e ferramentas.

```
make regression
  -> Python: referências, vetores e programas
  -> GCC/binutils: firmware C + assembly -> imagem da ROM
  -> Icarus: RTL + testbench -> executável de simulação
  -> vvp: estímulos, comparações e resultado PASS/erro
  -> Verilator: análise estática do RTL
  -> logs, manifesto SHA-256 e matriz da ISA
```

Os geradores usam sementes fixas (`0xC005` para vetores e `4711` para programas), permitindo repetir os estímulos. O modelo Python calcula resultados por inteiros/bytes; o hardware chega ao resultado por seus circuitos. O firmware compilado acrescenta uma origem de instruções independente do codificador dos geradores. Os scripts usam a biblioteca padrão de Python; ferramentas HDL e GCC são dependências do sistema, verificadas por `make doctor`.

O laboratório em oito aulas acompanha os arquivos reais, de testes unitários à preparação física. Ele documenta a aprendizagem e a reprodução do processo. Repetir uma aula é didático; uma alteração funcional exige nova regressão, independentemente de a aula correspondente já ter sido estudada.

2. Instruções Suportadas

2.1 Cobertura por categoria

A tabela segue a ordem e as quantidades do Guia de Submissão [R1, pp. 2–3]. A coluna baseline conta instruções integradas e verificadas; a confirmação do CRC experimental não está incluída. Nenhuma linha implica aprovação pelo firmware oficial, que ainda não foi executado neste projeto.

Categoria	Esperadas	Baseline	Cobertura
Arithmetic and Logic (Register)	10	10	100%
Arithmetic and Logic (Immediate)	9	9	100%
Load	5	5	100%
Store	3	3	100%
Branch	6	6	100%
Jump	2	2	100%
Upper Immediate	2	2	100%
Sistema / Sincronização	3	0	0%
Multiplication	4	4	100%
CRC	3	0	0%
Total	**47**	**41**	**87,23%**

2.2 Lista de instruções

Categoria	Instruções implementadas na baseline
Register	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
Immediate	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI
Load	LB, LH, LW, LBU, LHU
Store	SB, SH, SW
Branch	BEQ, BNE, BLT, BGE, BLTU, BGEU
Jump	JAL, JALR
Upper Immediate	LUI, AUIPC
Multiplication	MUL, MULH, MULHSU, MULHU

O perfil experimental acrescenta **CRCB, CRCH e CRCW**: três operações testadas no contrato CMOS. Somadas às 41 da baseline são 44 operações funcionais no perfil experimental (44/47 = 93,62%), sem equivaler a 44 instruções de conformidade oficial confirmada. A tabela principal permanece 41/47.

2.3 Critério de contagem e rastreabilidade

A contagem exige decode correto, execução integrada e comparação automática. Um teste isolado da ALU não basta para contabilizar uma instrução do processador. A matriz `docs/isa/coverage.csv` contém opcode, funct3, funct7, módulo, testes, regressão, firmware oficial, marco Git e evidência por instrução. Há versões Markdown e Excel para leitura. As tabelas seguintes transcrevem as codificações dessa matriz; os campos são apresentados em binário.

O traço em funct3/funct7 significa que esse campo não é discriminador daquela instrução. Em operações com imediato, esses bits podem pertencer ao imediato; não devem ser forçados a zero. Em SLLI/SRLI/SRAI, funct7 representa os bits superiores usados para distinguir as operações, e o deslocamento usa cinco bits.

2.4 Codificação — aritmética e imediatos

Instrução	Opcode [6:0]	Funct3 [14:12]	Funct7 [31:25]
ADD	0110011	000	0000000
SUB	0110011	000	0100000
SLL	0110011	001	0000000
SLT	0110011	010	0000000
SLTU	0110011	011	0000000
XOR	0110011	100	0000000
SRL	0110011	101	0000000
SRA	0110011	101	0100000
OR	0110011	110	0000000
AND	0110011	111	0000000
ADDI	0010011	000	—
SLTI	0010011	010	—
SLTIU	0010011	011	—
XORI	0010011	100	—
ORI	0010011	110	—
ANDI	0010011	111	—
SLLI	0010011	001	0000000
SRLI	0010011	101	0000000
SRAI	0010011	101	0100000
LUI	0110111	—	—
AUIPC	0010111	—	—

As operações SLT/SLTI usam comparação com sinal; SLTU/SLTIU, sem sinal. O imediato I é estendido com sinal inclusive em SLTIU. Deslocamentos por registrador usam os cinco bits inferiores de rs2. SUB e SRA são diferenciadas por funct7; SRAI preserva essa distinção na codificação imediata.

LUI coloca o imediato U nos bits altos do destino; AUIPC soma esse imediato ao PC da instrução. PASS_B é operação interna da ALU e não acrescenta uma instrução à tabela de cobertura.

2.5 Codificação — memória, controle de fluxo e multiplicação

Instrução	Opcode [6:0]	Funct3 [14:12]	Funct7 [31:25]
BEQ	1100011	000	—
BNE	1100011	001	—
BLT	1100011	100	—
BGE	1100011	101	—
BLTU	1100011	110	—
BGEU	1100011	111	—
JAL	1101111	—	—
JALR	1100111	000	—
LB	0000011	000	—
LH	0000011	001	—
LW	0000011	010	—
LBU	0000011	100	—
LHU	0000011	101	—
SB	0100011	000	—
SH	0100011	001	—
SW	0100011	010	—
MUL	0110011	000	0000001
MULH	0110011	001	0000001
MULHSU	0110011	010	0000001
MULHU	0110011	011	0000001

2.6 Codificação — extensão experimental

Instrução	Opcode [6:0]	Funct3 [14:12]	Funct7 [31:25]
CRCB	0110011	000	1000000
CRCH	0110011	001	1000000
CRCW	0110011	010	1000000

Os três códigos CRC constam do Block Guide [R1, pp. 12 e 14–15]. O contrato algorítmico completo foi escolhido internamente e é descrito na seção 4.3. Os nomes e códigos das três instruções de sistema/sincronização não foram inferidos apenas pela quantidade indicada no guia; permanecem pendentes.

3. Datapath

3.1 Caminho de dados e controle

A Figura 1 está dividida em três painéis complementares: execução, memória/PC e controle. Os desenhos foram conferidos com o RTL em 17/10/2026. Uma rede repetida pelo mesmo nome representa o mesmo sinal; as ligações entre painéis usam essa convenção para manter a leitura no tamanho de página do relatório.

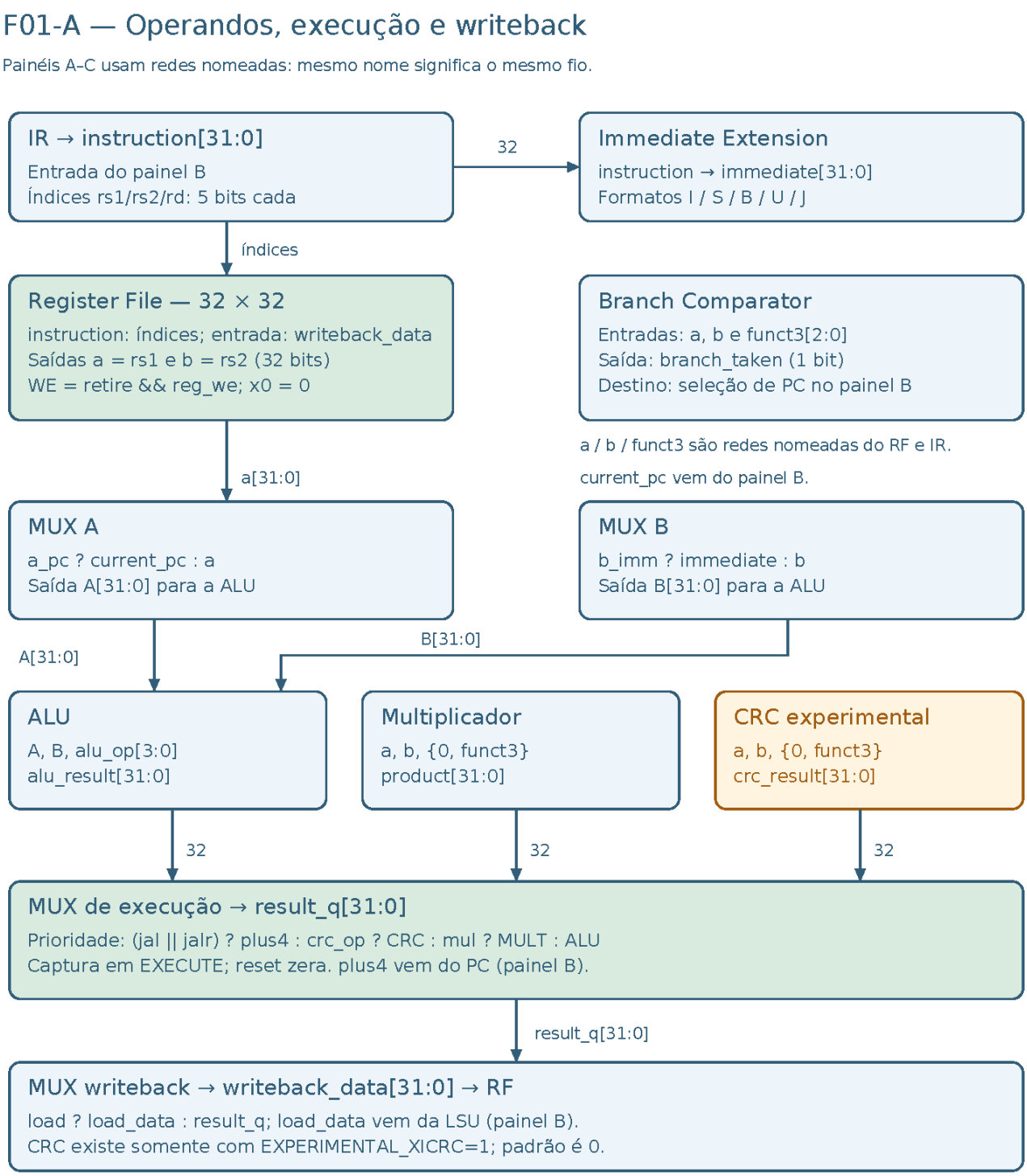


Figura 1-A — Operandos, execução e retorno ao banco de registradores. ALU, MULT e CRC recebem os sinais indicados; o CRC só existe com o parâmetro experimental habilitado. Fontes vetoriais e instruções de reprodução estão em [R3].

3.1 Caminho de dados e controle — memória e PC

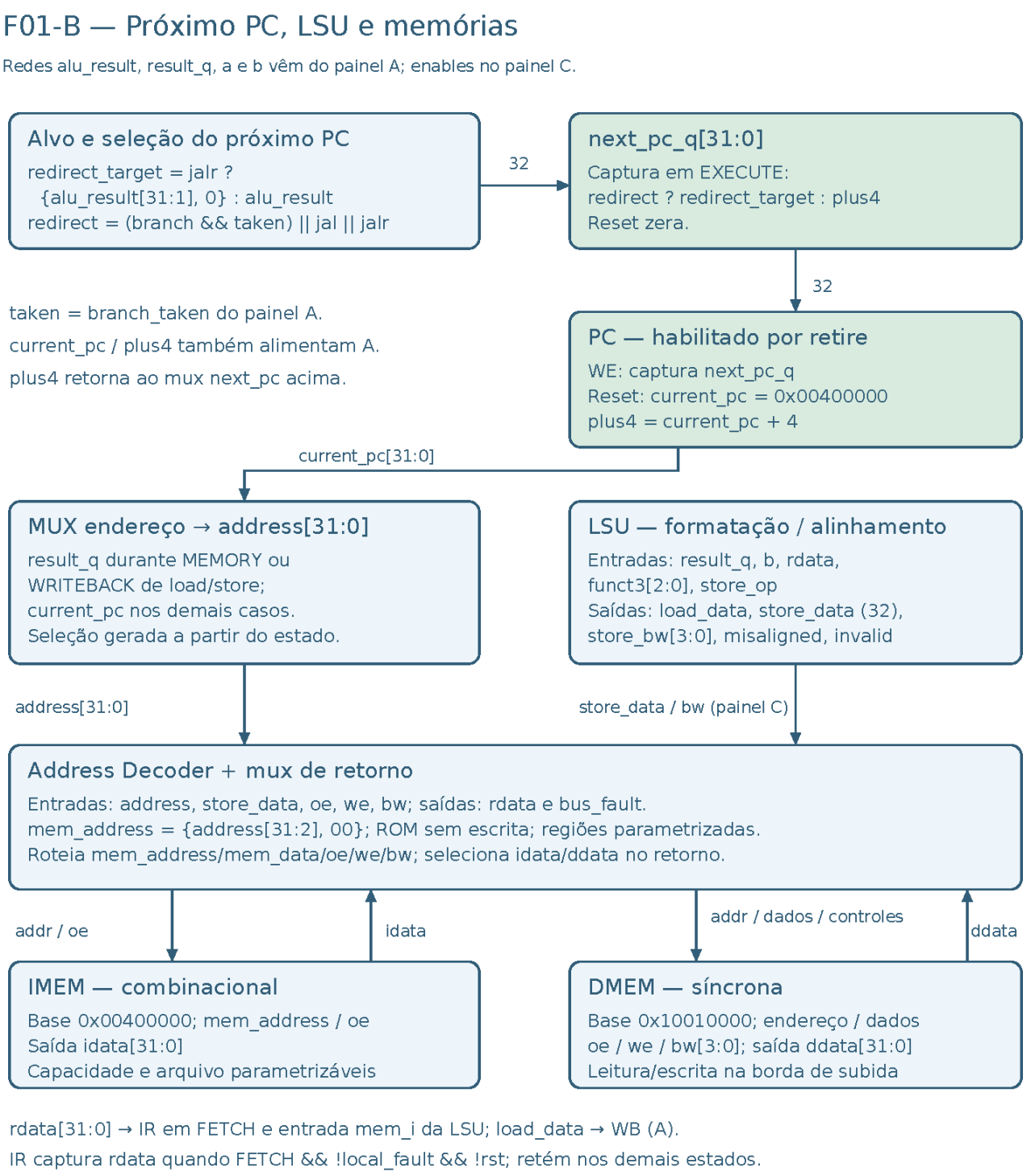


Figura 1-B — Endereçamento e retorno de memória. `alu_result`, `result_q`, a e b são as redes do painel A. `branch_taken` determina o branch junto ao decode. A LSU recebe a palavra lida, posiciona stores e produz o dado de load. O Address Decoder roteia os pedidos, protege a ROM e seleciona o retorno. O PC atualiza na aposentadoria e o IR captura somente em FETCH válido.

3.1 Caminho de dados e controle — sequenciamento e enables

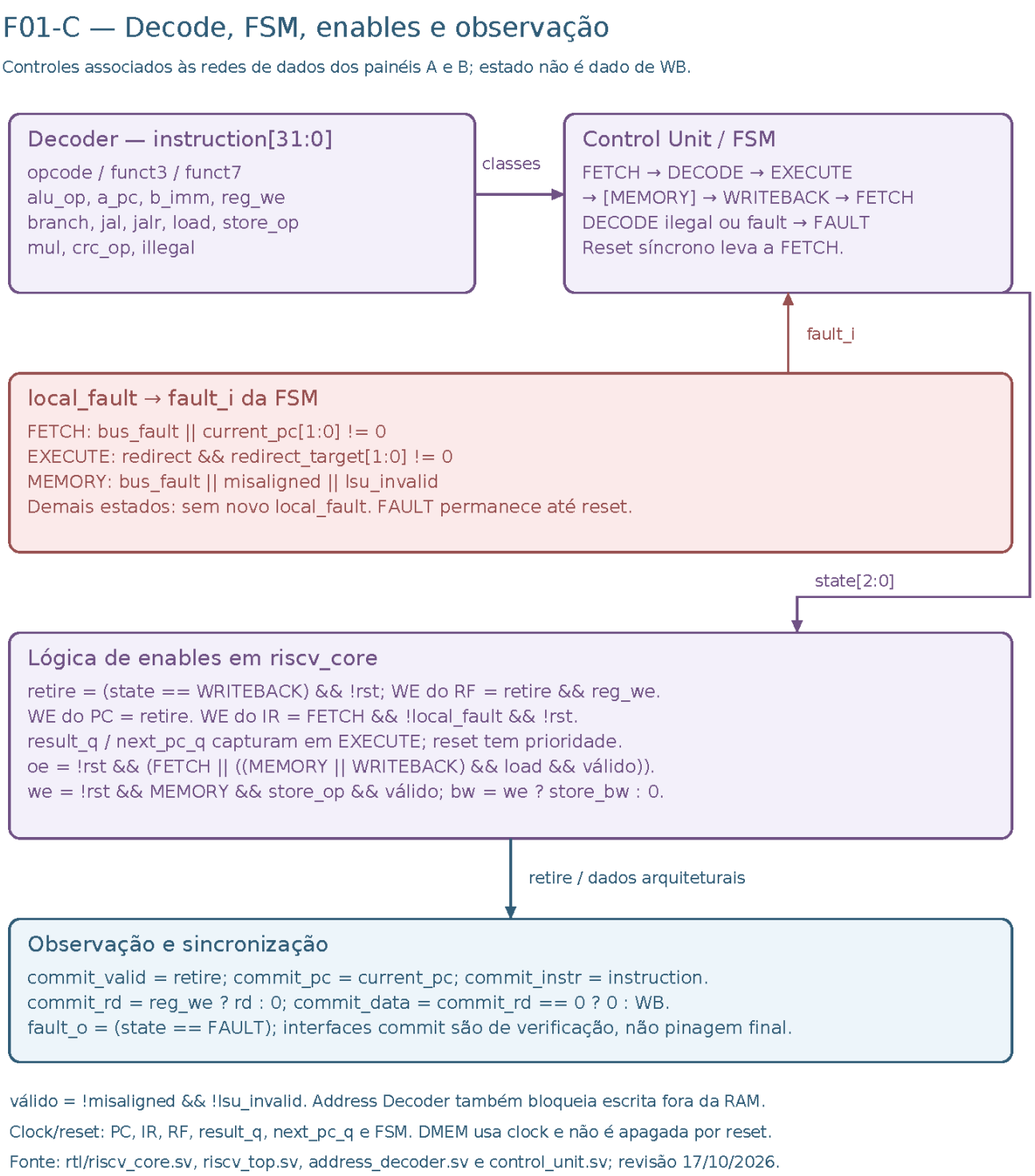


Figura 1-C — Controle associado aos dois painéis de dados. O decoder gera seleções e classes; a FSM recebe classificação/erros; a lógica do core produz enables e controles do barramento. A política FAULT é um diagnóstico CMOS. Os sinais `commit_*` são observação para verificação, sem definir a pinagem final. Os desenhos representam a implementação atual. Diagramas por módulo feitos no ChipInventor e revisão da interface final continuam pendentes.

3.2 Sequenciamento multiciclo

Estado	Operação	Atualização / próximo estado
FETCH	Lê instrução usando PC	IR captura; segue a DECODE
DECODE	Valida campos e lê operandos	EXECUTE; código inválido leva a FAULT
EXECUTE	Calcula resultado, alvo ou endereço	Guarda result_q/next_pc_q; MEMORY se load/store
MEMORY	Acesso à DMEM ou leitura de constante na IMEM	Segue a WRITEBACK se acesso válido
WRITEBACK	Retorno ao RF quando aplicável	Atualiza PC e volta a FETCH
FAULT	Bloqueia progresso	Permanece até reset

Instruções sem acesso a dados usam quatro ciclos; loads/stores usam cinco. O estado MEMORY atende a latência da DMEM síncrona. A escrita de store ocorre somente nesse estado; WRITEBACK aposenta a instrução sem repetir a escrita. PC permanece estável durante a instrução e é atualizado na aposentadoria. Essa escolha preserva o endereço da instrução para AUIPC, branches e verificação.

Reset suprime escritas, reinicia o controle e zera os registradores internos previstos no contrato. DMEM não é apagada pelo reset. Endereço inválido, acesso desalinhado, escrita na ROM ou instrução não suportada geram diagnóstico FAULT. Esse mecanismo não implementa traps privilegiados, CSRs ou as três instruções de sistema ainda pendentes.

3.3 Memórias e interface

Configuração	IMEM	DMEM
Referência de integração	0x00400000–0x007fffff; 4 MiB	0x10010000–0x10011fff; 8 KiB
simple_system experimental	0x00400000–0x004007ff; 2 KiB	0x10010000–0x100103ff; 1 KiB

Capacidades são parâmetros; as bases permanecem as do guia. Endereços externos são em bytes. O índice interno de palavra é `(endereço - base) >> 2`. O retorno da IMEM é combinacional; a DMEM possui leitura e escrita síncronas. A organização de bytes é little-endian. A IMEM também fornece constantes para loads; escritas nela são rejeitadas. O decoder de endereços impede aliases fora da capacidade configurada.

A LSU usa os bits baixos do endereço para selecionar bytes, estender sinal ou zeros e posicionar stores. A máscara `bw[3:0]` preserva bytes não selecionados; `bw[0]` corresponde a bits [7:0]. Halfwords exigem alinhamento de dois bytes e palavras, de quatro. Leituras e escritas são habilitadas por `oe` e `we`.

A memória comportamental do RTL não comprova o uso de uma macro SRAM física. No experimento de síntese, ROM torna-se lógica constante e RAM torna-se registradores/multiplexadores. A escolha de memória para o layout final deve ser avaliada em conjunto com capacidade, área e temporização.

4. Módulos e Testbenches

4.1 Blocos básicos

Os nomes abaixo correspondem aos arquivos `rtl/<nome>.sv` e às documentações `docs/modules/<nome>.md`. As bancadas individuais estão em `tb/unit/`; as conexões progressivas, em `tb/integration/`. Os resultados citados são os da rodada registrada [R4–R6]. Diagramas Mermaid existentes documentam o trabalho, mas **as imagens por módulo feitas no ChipInventor ainda devem ser produzidas**.

ALU — alu. Circuito combinacional com entradas A/B de 32 bits, seletor de 4 bits e resultado de 32 bits. Executa PASS_B, ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT e SLTU. `tb_alu` compara 18.304 vetores: zero, extremos, resultados modulares, signed/unsigned, shifts 0/31 e seletores inválidos. `tb_rf_alu` verifica operandos e retorno ao banco; `tb_datapath` acrescenta seleção de operandos. Resultado registrado: PASS.

Banco de registradores — regfile. Possui 32 palavras de 32 bits, duas portas de leitura combinacional e uma escrita síncrona habilitada. x0 sempre retorna zero e ignora escrita. `tb_regfile` verifica destinos, 1.024 pares de leitura, enable e reset. `tb_rf_alu` exercita dependências reais de dados. Resultado registrado: PASS.

Extensão de imediatos — imm_ext. Reconstrói os formatos I, S, B, U e J a partir de uma instrução de 32 bits. `tb_imm_ext` compara 5.102 vetores, incluindo extremos, bits individuais e extensão de sinal. `tb_ir_imm` verifica que o imediato corresponde à instrução efetivamente capturada. Resultado: PASS.

Contador de programa — pc. Mantém o endereço de 32 bits e fornece PC+4. Reset e write enable controlam a atualização. `tb_pc` verifica retenção, sequência, wrap e reset prioritário; `tb_pc_alu` integra cálculo de alvos e `tb_branch_pc` testa tomada de desvio. Resultado: PASS.

Registrador de instrução — ir. Captura a palavra em FETCH quando habilitado e a mantém nos estados seguintes. `tb_ir` verifica captura/retenção e reset; `tb_fetch` integra PC, ROM e IR, e `tb_ir_imm` conecta a extração de imediato. Resultado: PASS.

Memória de instruções — imem. ROM parametrizável, inicializada por imagem hexadecimal completa ou por zeros quando não há arquivo. `tb_imem` verifica conteúdo, enable, limites e alinhamento; `tb_fetch` verifica a sequência de busca. A ROM também fornece constantes ao firmware. Resultado: PASS.

Comparador de branch — branch_comp. Compara rs1/rs2 conforme funct3, usando igualdade, ordem com sinal ou sem sinal. `tb_branch_comp` compara 9.152 vetores; `tb_branch_pc` e `tb_isa` verificam os efeitos sobre PC, inclusive desvio tomado, não tomado e alvos negativos. Resultado: PASS.

Figuras a incorporar: F02 ALU, F03 RF, F04 Immediate, F05 PC, F06 IR, F07 IMEM e F08 Branch Comparator, exportadas do ChipInventor com pinos legíveis.

4.2 Memória, decodificação e controle

LSU — lsu. Recebe endereço efetivo, dado de store, palavra lida, funct3 e seleção load/store. Produz dado de load estendido, dado de store reposicionado, byte write e indicação de erro. `tb_lsu` compara 8.960 vetores para todas as larguras, offsets, extensão e desalinhamento. `tb_lsu_rf` verifica retorno ao RF; `tb_lsu_dmem` verifica escritas parciais e leituras. Resultado: PASS.

Memória de dados — dmem. Array de palavras de 32 bits com leitura síncrona e escrita independente dos quatro bytes. Conteúdo não é limpo por reset. `tb_dmem` exercita 256 combinações de posição/máscara, preservação dos bytes, retenção e limites, incluindo o comportamento read-first. Integração com LSU e barramento é verificada separadamente. Resultado: PASS.

Decodificador de endereços — address_decoder. Seleciona IMEM/DMEM, encaminha endereço e controles, multiplexa retorno e informa acessos inválidos. Não permite escrita na IMEM. `tb_address_decoder` cobre 768 combinações de faixa, pedido e máscara. `tb_memory_bus` integra as memórias; `tb_memory_faults` verifica falhas e bloqueio de efeitos. Resultado: PASS.

Decoder de instruções — decoder. Recebe a palavra de instrução e gera seletores de ALU, operandos, writeback e classes de operação. Só reconhece CRC quando habilitado pelo parâmetro experimental. `tb_decoder` varre 131.072 combinações de opcode/funct3/funct7 em cada perfil, incluindo códigos inválidos. Essa varredura cobre campos de decode, não todos os 2^32 valores de instrução. `tb_core_smoke`, `tb_isa` e `tb_core_crc` verificam execução. Resultado: PASS.

Unidade de controle — control_unit. FSM síncrona que seleciona FETCH, DECODE, EXECUTE, MEMORY, WRITEBACK ou FAULT. Entradas de classificação e erro definem a transição; o core deriva os enables do estado. `tb_control_unit` verifica sequências e reset. `tb_core_faults` verifica erro de decode/controle e recuperação; `tb_memory_faults` verifica erros de barramento. Resultado: PASS.

Os testes de memória conferem tanto o dado observado quanto o efeito físico no modelo de RAM: o teste integrado contabiliza **45 stores** e compara **12 bytes finais**. Isso permite detectar escritas repetidas ou bytes alterados fora da máscara, mesmo quando o registrador de retorno parece correto.

Os casos de falha incluem instrução ilegal e acessos inválidos. Há 12 casos de decode/controle e 12 de memória na documentação de verificação. O diagnóstico retido até reset evita que um erro seja confundido com término bem-sucedido.

Figuras a incorporar: F09 LSU, F10 DMEM, F11 Address Decoder, F12 Decoder e F13 Control Unit, com conexões de controle e larguras explícitas.

4.3 Multiplicação, CRC e sistemas integrados

Multiplicador — multiplier. Seleciona operandos com ou sem sinal e retorna os 32 bits inferiores ou superiores do produto. Implementa MUL, MULH, MULHSU e MULHU. `tb_multiplier` compara 18.304 vetores, incluindo zero, um, -1, extremos e combinações mistas. `tb_mult_rf` verifica retorno ao banco; `tb_isa` verifica as quatro instruções no núcleo. Resultado: PASS.

CRC — crc. Implementação combinacional experimental de CRC-16, polinômio `0x1021`, sem reflexão e sem XOR final. rs1[15:0] fornece o acumulador; CRCB/CRCH/CRCW consomem 1/2/4 bytes baixos de rs2, do byte menos significativo para o mais significativo, processando bits de cada byte do MSB ao LSB. O resultado de 16 bits é estendido com zeros. A semente é fornecida pelo software; não existe estado oculto de inicialização.

`tb_crc` compara 18.304 vetores com o modelo Python baseado em `binascii.crc_hqx`. O vetor ASCII 123456789, iniciado em `0xffff`, produz `0x29b1`. `tb_core_crc` verifica 2.127 instruções CRC em um programa de 32.147 aposentadorias, incluindo destinos sobrepostos aos operandos, x0, memória e reset. Resultado: PASS **no contrato experimental** [R5, R8]. A implementação não acrescenta um estado de execução iterativa; seu custo físico e adequação ao contrato oficial ainda precisam ser avaliados.

Núcleo — riscv_core. Integra os blocos da seção 3 e expõe um barramento compartilhado para instruções/dados. `tb_core_smoke` verifica a execução mínima; `tb_isa` compara a aposentadoria de 76.409 instruções com modelo de referência. `tb_core_faults` e `tb_core_crc` ampliam a verificação. Resultado: PASS dentro dos perfis implementados.

Top de integração — riscv_top. Liga núcleo, decoder de endereços e memórias parametrizáveis. É exercitado por `tb_isa`, `tb_firmware` e testes de falhas de memória. Seus sinais de observação facilitam verificação; UART/GPIO e pinagem final da submissão permanecem em aberto. Resultado: PASS funcional no escopo.

Sistema experimental — simple_system. Usa ROM 2 KiB/RAM 1 KiB e monitora a primeira escrita SW completa em `0x100103f0`. `done_o` registra término; `success_o` indica a assinatura `0x600dc0de`; `fault_o` informa erro do núcleo. Os sinais ficam retidos até reset; firmware permanece em loop após a assinatura. `tb_simple_system` verifica sete cenários de boot/reset/assinatura/falha. `scripts/synth/tb_simple_netlist.sv` verifica dois boots observando só os pinos. Resultado: PASS em RTL e netlist genérica [R6].

Figuras a incorporar: F14 Multiplicador, F15 CRC, F16 Core, F17 Top e F18 simple_system. O último deve ser identificado como experimento, sem confusão com a interface final requerida pela competição.

4.4 Estratégia de teste e reprodução

A verificação é autoverificável: o testbench encerra com erro ao encontrar resultado divergente, enquanto uma sequência completa produz PASS. A inspeção de waveform auxilia o diagnóstico, mas não é o único critério de aprovação.

Nível	Comando na raiz	Escopo
Diagnóstico	make doctor	Disponibilidade e versões das ferramentas
Um bloco	make tb TB=tb_alu	Gera vetores, compila e simula uma bancada
Unitários	make unit	14 bancadas de módulos
Integração	make integration	17 bancadas de conexões, core e firmware
Rápido	make smoke	Quatro integrações para detectar quebra estrutural
Estático	make lint	Verilator, dois tops elaborados separadamente
Regressão	make regression	31 bancadas, lint, cobertura e manifesto
Firmware	make run	Build e execução do smoke próprio
Netlist	make simple-synth	Síntese genérica e simulação adicional

Executar um comando por vez: os alvos compartilham `build/` e `results/logs/`. `make -n <alvo>` mostra a receita. `make test` executa os testbenches sem lint; para uma rodada de fechamento deve-se usar `make regression`.

O executor rejeita erro de processo, timeout, diagnóstico de compilação ou falta de PASS. O limite é de 120 segundos por processo da regressão; síntese usa limite próprio de 180 segundos por processo. O manifesto registra estado, lista de bancadas, lint, cobertura e hashes SHA-256. Uma rodada interrompida ou marcada RUNNING não equivale a PASS.

O lint mantém checagens de largura, latches e loops combinacionais; supressões se restringem a nomes de arquivo e sinais/parâmetros sem uso. O teste integrado compara PC, instrução, destino, valor escrito e estado do banco na aposentadoria. Programas auxiliares que preparam operandos também entram nessa contagem.

4.5 Evidências e limites

A rodada [R6] contém **31 TBs PASS e lint PASS**. A rodada anterior dedicada ao CRC [R5] contém 30 TBs; a diferença é a inclusão posterior de `tb_simple_system`. O teste da netlist é adicional e não entra na contagem das 31 bancadas RTL.

Os manifestos identificam os arquivos testados naquela execução. Mudanças posteriores de comentários e documentação não constituem nova execução; a versão final deve receber nova regressão e manifesto, associados ao commit de entrega. Logs correntes podem ser sobrescritos por testes individuais, por isso as rodadas experimentais têm cópias identificadas em `results/experiments/`.

Os testes não constituem prova formal, cobertura exaustiva de operandos nem certificação da ISA. A cobertura de 41/47 é uma matriz funcional por instrução, não uma medição de cobertura de código HDL. Firmware oficial, figuras ChipInventor e evidências físicas ainda faltam para a entrega completa.

5. Resultados OpenLane

5.1 Estado do entregável físico

Fluxo físico do processador ainda não executado. Área total, densidade e imagem do GDSII final, exigidas por [R1, p. 4], não estão disponíveis nesta versão. Não há `config.json`, GL final nem GDSII do processador para submissão.

Resultado exigido / registro	Estado nesta versão	Preenchimento final
Área total	Não medida	Valor, unidade e relatório de origem
Densidade final	Não medida	Valor e definição usada pelo fluxo
Imagem do GDSII	Ausente	Figura F19 com escala e identificação do top
GDSII, GL e config.json	Ausentes	Arquivos da mesma execução final
Commit, ferramentas e PDK	Ambiente conhecido; run do core ausente	Identificar a execução final
Timing, DRC/LVS e antena	Sem avaliação do processador	Registrar resultados e limitações

As três primeiras linhas correspondem ao conteúdo expressamente solicitado para esta seção. Timing e verificações físicas são registros técnicos do projeto que permitirão interpretar a validade do layout e suas limitações.

5.2 Preparação do ambiente

O ambiente Docker UNIC-CASS contém LibreLane 3.0.0rc1 e PDK SKY130A. O teste embutido do ambiente chegou a 80/80 etapas, com DRC/LVS/antena PASS [R7]. O circuito desse teste é um exemplo da ferramenta. Seu GDSII não é o GDSII do CMOS RISC-V e seus resultados não preenchem a tabela física do processador. A utilização dessa versão do fluxo na entrega deve ser conferida com as orientações da organização sobre OpenLane.

5.3 Síntese lógica preparatória do simple_system

A síntese genérica com Yosys 0.39+124 (`d73f71e81`) gerou **32.922 células genéricas**. Não foram usados PDK ou restrições de timing. Memórias foram mapeadas, não restaram atributos de inicialização de registradores e as saídas de resultado não se tornaram constantes. A netlist executou o firmware em dois boots com RAM inicialmente indefinida [R6].

Foi identificada uma diferença entre simulação RTL e síntese: zerar toda a IMEM antes de `$readmemh` resultava em ROM zerada na netlist. A inicialização passou a selecionar arquivo OU zeros, e o gerador fornece uma imagem completa. O teste do firmware na netlist verificou a correção. Esse caso demonstra por que a aprovação apenas em RTL não encerra a validação de implementação.

A ROM foi resolvida em constantes; trocar o programa exige ressíntese nesse experimento. A RAM foi mapeada em registradores/muxes. A contagem genérica não é área em micrômetros quadrados nem comprovação de viabilidade física. O avanço futuro deve definir memória, top, restrições e configuração, executar o fluxo e registrar área/densidade, warnings, etapa alcançada e arquivos finais.

6. Firmware Demo

6.1 Firmware próprio da baseline

O programa em `firmware/smoke/` verifica a execução de código C/assembly sem sistema operacional. `start.S` configura a pilha, copia `.data` da ROM para RAM, zera `.bss` e chama `main`. `link.ld` posiciona texto, constantes, dados e pilha nas regiões do sistema. `main.c` verifica inicialização, chamadas, variáveis locais, loops, multiplicação e acessos parciais à memória.

O script `scripts/sim/build_firmware.py` invoca GCC/binutils para `rv32i_zmmul` e ABI `ilp32`, produz executável, disassembly e imagem hexadecimal completa da ROM. A instrução comprimida não é utilizada. O firmware pode exercitar apenas parte da ISA em cada trecho; sua aprovação não substitui a matriz individual das 41 instruções.

O retorno de `main` é armazenado pelo startup em `0x10011ff0`. A assinatura `0x600dc0de` indica sucesso; valores `0xdead0001` a `0xdead0006` identificam classes de erro dos autotestes. Depois disso o processador permanece em loop. O testbench reconhece o término pela escrita, sem exigir uma instrução de sistema ainda não especificada.

```
make run
make wave
```

O segundo comando também gera `results/waveforms/smoke.vcd`. A execução isolada de 17/10/2026 registrou em `results/experiments/firmware-wave/tb_firmware.wave.log`:

```
PASS firmware: signature 600dc0de, 598 retirements, 2579 cycles
```

A execução própria terminou com **598 instruções aposentadas e 2.579 ciclos**. Esse resultado caracteriza esse programa e a configuração simulada; não é frequência máxima, benchmark geral ou aprovação do firmware oficial.

6.2 Waveform do firmware próprio

As figuras F20-A/B foram extraídas do VCD real gerado por `make wave` em uma cópia isolada do projeto, com o RTL do commit `7ddc6bf`. O inventário SHA-256, versões das ferramentas, logs e VCD comprimido estão em [R10]. As imagens são exportações programáticas dos valores capturados, não desenhos de resultados esperados. A escala de simulação é 1 ps por tick; o clock do testbench tem período de 10 ns. Isso não mede a frequência máxima do circuito físico.

6.2 Waveform — boot

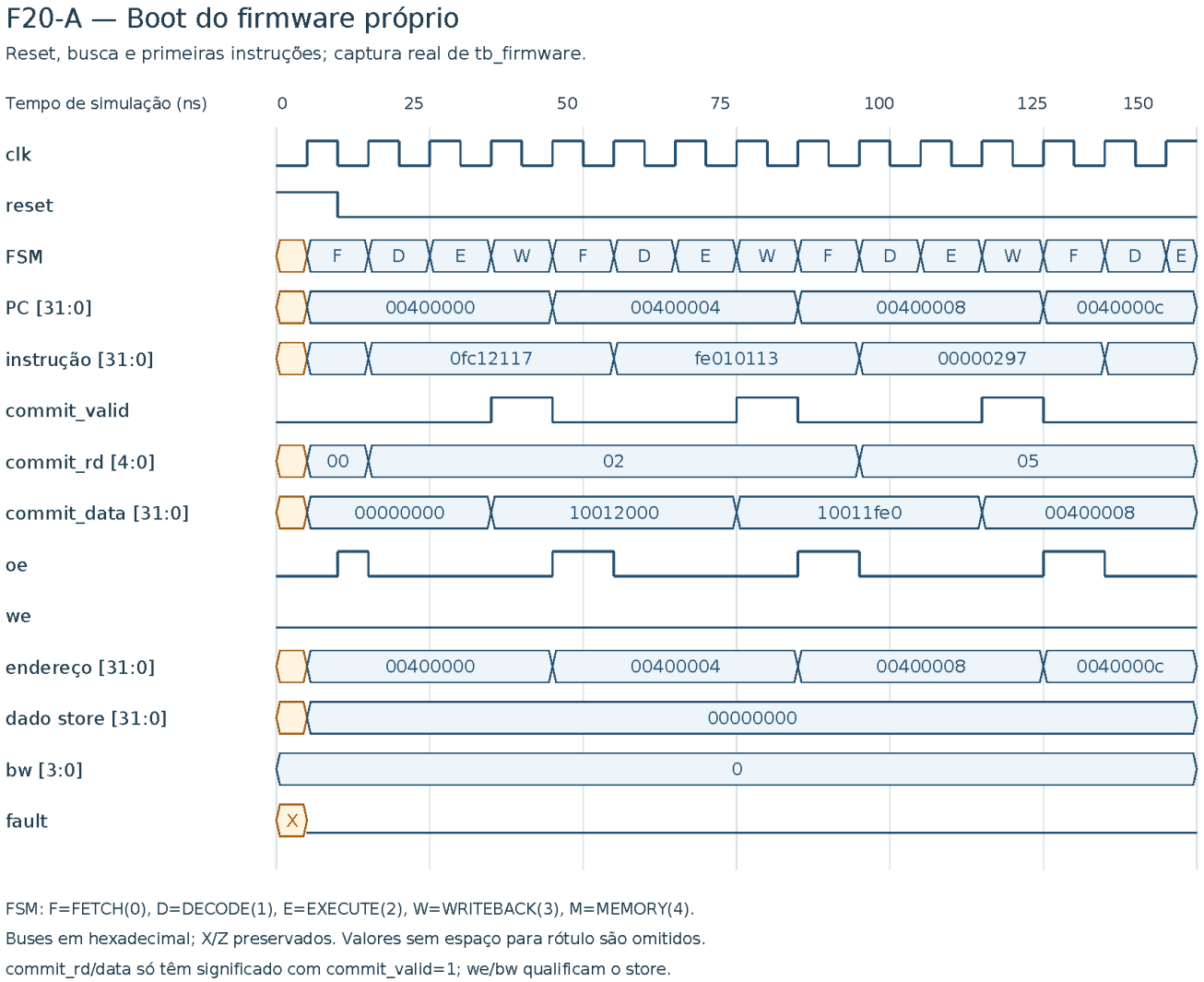


Figura F20-A — Janela de 0 a 150 ns da execução própria de 17/10/2026. O reset é aplicado na primeira borda de subida, em 5 ns, e liberado em 10 ns. Após isso, a FSM percorre busca, decode, execução e writeback. O PC inicia em `0x00400000` e avança por palavras; `commit_valid` indica as aposentadorias. Os valores X anteriores à primeira borda de reset são preservados na figura. Os buses são exibidos em hexadecimal; intervalos curtos podem não ter rótulo. `commit_rd` e `commit_data` devem ser interpretados quando `commit_valid=1`.

A captura mostra as primeiras instruções do startup, que estabelece a pilha e prepara a inicialização de dados. O VCD completo cobre a execução até o teste final de assinatura; esta figura é apenas uma janela inicial para leitura.

6.2 Waveform — escrita da assinatura

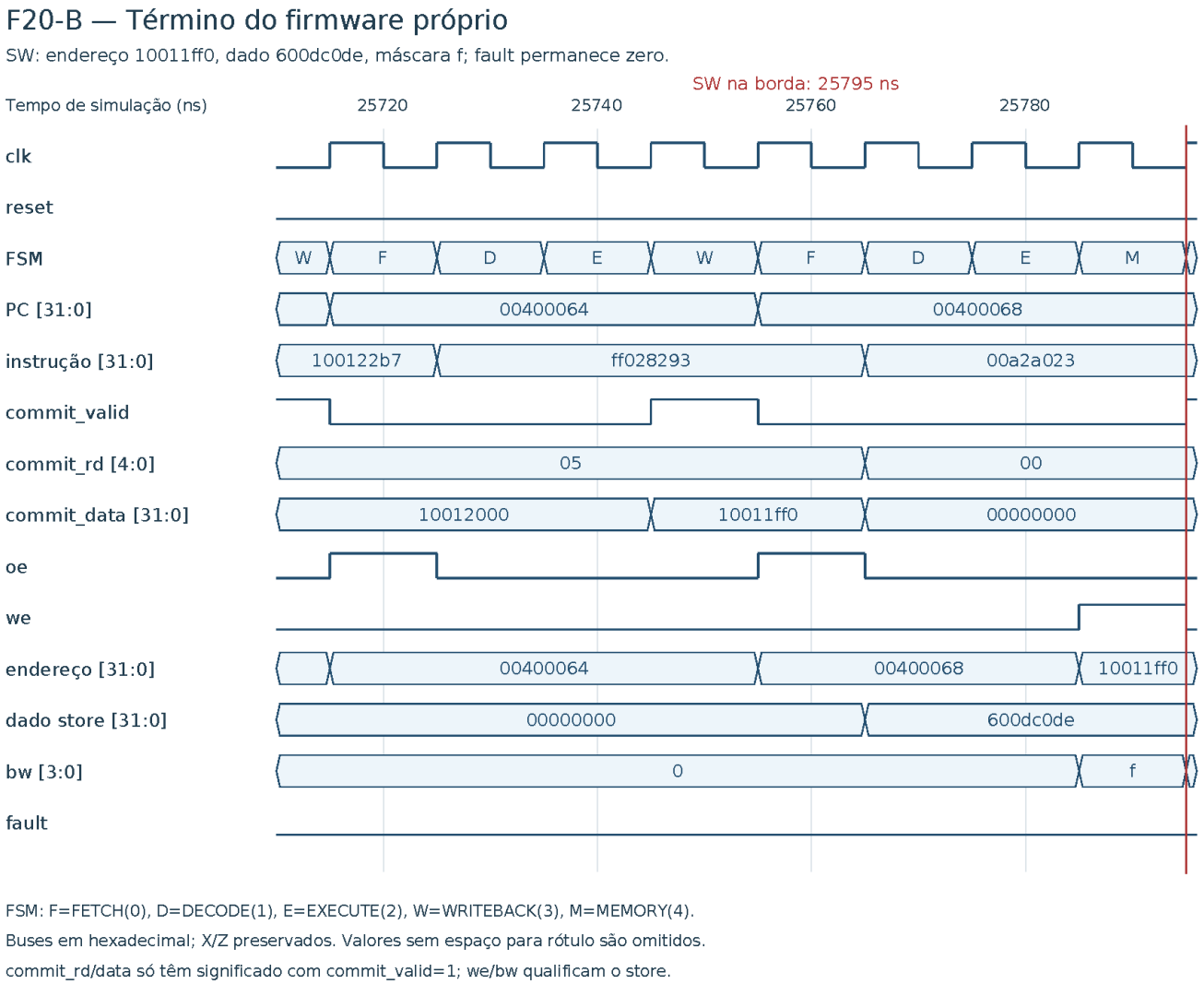


Figura F20-B — Janela de 25.710 a 25.796 ns da mesma execução. A instrução SW `0x00a2a023`, no PC `0x00400068`, escreve `0x600dc0de` no endereço `0x10011ff0`. Durante MEMORY, `we=1` e `bw=0xf`; a escrita ocorre na borda de subida marcada em vermelho, **25.795 ns**. O sinal fault permanece zero. O testbench confere a palavra efetivamente armazenada e encerra a simulação um nanossegundo depois, com o log PASS transcrito na seção 6.1.

A contagem de 598 aposentadorias corresponde às instruções já concluídas antes dessa escrita. A instrução SW final ainda não alcançou a borda de sua aposentadoria quando o testbench termina. A análise do VCD usa os valores anteriores à borda para conferir a transação síncrona e reproduziu tanto os 598 commits quanto os 2.579 ciclos do testbench.

Essas imagens e logs são evidência do firmware próprio. O firmware oficial precisa de sua própria execução, waveform e protocolo de conclusão, conforme seção 6.4. Não houve execução de regressão completa ou de fluxo físico nesta rodada.

6.3 Firmware do sistema experimental

`firmware/simple/` usa a mesma sequência de inicialização em um mapa reduzido, com pilha inicial em `0x100103e0` e assinatura em `0x100103f0`. A imagem da rodada de referência ocupa 608 dos 2.048 bytes de ROM e reserva 32 bytes de BSS. O linker separa dados, pilha e assinatura; a RAM não depende de reset para inicializar o estado do software.

O programa verifica `.data`, `.bss`, pilha, MUL, escritas de byte/halfword e CRCB/CRCH/CRCW. As instruções customizadas são emitidas com diretivas `.insn r` que indicam opcode/funct3/funct7. Três formas de agrupar 123456789 em bytes, meias palavras e palavras chegam ao CRC `0x29b1` com semente `0xffff`.

```
make simple-system
make simple-synth
```

Saídas das evidências [R6]:

```
PASS simple_system: 7 boot/reset/signature/fault checks,
provisional CRC firmware
PASS simple_netlist: two boots with ROM constants and uninitialized RAM
```

O primeiro log foi quebrado em duas linhas apenas para leitura. O monitor só encerra por SW completa; uma escrita parcial não basta. Reset permite novo boot com RAM contendo dados da execução anterior. O teste de netlist observa pinos, verificando término e sucesso sem acessar internamente ROM/RAM.

6.4 Firmware oficial e fechamento

Ainda não executado no projeto. O guia informa distribuição pela plataforma ChampionCHIP e avaliação até a etapa alcançada [R1, p. 4]. Arquivo, versão, protocolo de término e evidência de execução não constam do repositório nesta revisão. Os resultados próprios não recebem a pontuação do programa oficial.

Para completar esta seção, registrar origem e hash do firmware oficial, configuração de compilação/carga, commit RTL, etapa final atingida, instrução/PC em caso de falha, log do testbench e **Figura F21: waveform oficial**. A aprovação integral só deve ser declarada quando o protocolo oficial indicar conclusão sem erros. Após eventual correção, repetir os testes anteriores e a execução oficial.

O fechamento do projeto exige ainda confirmar o contrato Xicrc, implementar as três instruções de sistema especificadas, revisar interface final, executar a regressão da versão de entrega, incorporar diagramas ChipInventor e completar a seção física. Vídeo de 5–7 minutos no YouTube, GDSII, GL, configuração e código/TBs compõem os demais entregáveis exigidos pelo guia.

6.5 Referências e localização das evidências

As páginas de [R1] são posições físicas do PDF combinado de 24 páginas, não apenas a numeração reiniciada do Block Guide. Os caminhos locais referem-se à raiz do repositório. Na submissão, identificar o commit definitivo do pacote.

[R1] ChampionCHIP — Guia de Submissão e Block Guide, 2026, versão 1.0. Arquivo Champion-chip-guia-fase-2.pdf. Guia de Submissão: pp. 2–4; Block Guide: pp. 6–23. Leitura para esta versão: 15/10/2026. SHA-256: `3107f22f7153aca25ea8f472be4dff7474b5c6b9b5b1f0d490da190f5e85ab41`. Esta é a fonte normativa local consultada. Não se afirma que substitua retificações ou instruções adicionais da plataforma da competição.

[R2] Processo CMOS. AGENTS.md, decisões arquiteturais e questões oficiais pendentes. São regras e decisões internas; não equivalem ao regulamento da organização.

[R3] Arquitetura. Datapath, figuras e reprodução, interfaces, mapa de memória e documentação dos módulos.

[R4] Baseline e cobertura. Verificação, matriz CSV, matriz legível, manifesto histórico e log do firmware próprio. Tag RTL `isa-41-v1` (ea6bccd). Consultar a identificação da rodada antes de combinar logs de testes individuais com um manifesto anterior.

[R5] Experimento CRC. Registro e comandos, manifesto e logs dessa pasta; implementação introduzida em `05fd114`.

[R6] Sistema pequeno e síntese. Registro, manifesto RTL, resumo Yosys, log da netlist e contrato do sistema. Implementação introduzida em 2e87e0e; os hashes identificam o conteúdo efetivamente testado.

[R7] Ambiente físico. Registro do smoke do ambiente e documentação OpenLane/LibreLane. Essas evidências pertencem ao exemplo da ferramenta.

[R8] CRC provisório. Contrato CMOS e matriz experimental. As referências do algoritmo e o escopo da escolha interna constam desses documentos.

[R9] Reprodução. Makefile, instalação, roteiro de testes, síntese e laboratório em oito aulas. As instruções RV32I/Zmmul usam as fontes registradas em fontes arquiteturais.

[R10] Captura do firmware próprio — 17/10/2026. Registro e reprodução, inventário da execução, verificação do VCD e log do testbench. O VCD comprimido permite reproduzir as figuras sem executar uma nova simulação.

Estado editorial: redação atualizada com evidências disponíveis; autoria, figuras, resultados físicos e execução oficial devem ser completados e revisados antes de nomear esta versão como relatório final de submissão.